

Proposal of `[[uninitialized]]` attribute

P0632R0

ISO/IEC JTC1 SC22 WG21 2017-01-19

Jonathan Müller, jonathan.mueller@foonathan.net

Summary

The attribute `[[uninitialized]]` is proposed. It can be used to mark variables that are deliberately not initialized. This gives both humans and tools better information about the intent and allows guideline checks and warnings about not initialized variables.

Proposal

This paper proposes the standardization of a new attribute `uninitialized`. It is a hint for humans and tools that a variable was not initialized *on purpose*, and not an oversight.

There are two separate variants of the attribute: `[[unintialized]]` and `[[uninitialized(uninitialized-id-list)]]`.

The first version - `[[uninitialized]]` - may only appear on definitions of local variables. It simply marks this variable as not initialized and that it will be assigned later on. A variable marked `[[uninitialized]]` must be "initialized" using default initialization, where the selected constructor in case of a class type is trivial (i.e. it must not be initialized at all). Otherwise, the program shall be ill-formed, no diagnostic required.

The second version - `[[uninitialized(uninitialized-id-list)]]` - may only appear on definitions of constructors. Unlike the first version in this case the attribute has a list of comma-separated arguments. Those are the names of the member variables - same lookup rules as in the member initializer list - which are not initialized in this constructor. Like in the first version, the member variables which are marked `[[uninitialized]]`, must be "initialized" using trivial default initialization, otherwise the program shall be ill-formed, no diagnostic required.

Following the same philosophy as [P0068R0](#), this paper does not mandate any specific behavior. A conforming implementation may simply ignore the attribute completely. However, implementations are encouraged to provide an (optional) warning whenever a variable is not initialized and not marked with the attribute.

Note that this paper does not propose any changes to the consequence of reading uninitialized variables, whether marked with `[[uninitialized]]` or not.

Motivation

Uninitialized variables can be a big source of bugs. That is why coding standards such as the [C++ Core Guidelines](#) recommend that one shall "always initialize an object" [ES.20](#). However, there are rare cases where it is desirable to leave a variable uninitialized, as it will be initialized later.

For example, this is one exception given by the rule [ES.20](#):

```
constexpr int max = 8 * 1024;
int buf[max];
f.read(buf, max);
```

Assuming the read was successful, the original programmer knows that `buf` will have valid values before it is used somewhere else. But this information is not easily available to other programmers or automated guideline checkers. While deliberately leaving variables uninitialized should be commented in the source code, this still only helps humans reading the code.

The proposed `[[uninitialized]]` attribute provides a standardized way of expressing the intent of leaving a variable uninitialized. This informs both other programmers and tools that the initialization was not forgotten, but deliberately not done. Guideline checkers, which would normally issue a warning about

an uninitialized variable, can then use that attribute to silence it.

Not initializing a variable can be comparable with a fallthrough in a switch. Sometimes it is desired, but often it is an oversight. But compilers cannot issue a warning about missing initialization, because it can be intended. That is why the `[[fallthrough]]` attribute was added, `[[uninitialized]]` can serve the same purpose for missing variable initialization. Both make the implicit behavior more explicit and allow warnings when the default behavior is used.

Examples

Example 1: `[[uninitialized]]` on local variables

```
void func()
{
    // problematic: not initialized on purpose or oversight?
    int array[big_number];
    read(array, big_number);
}
```

With `[[uninitialized]]`:

```
void func()
{
    // OK: not initialized on purpose
    [[uninitialized]] int array[big_number];
    read(array, big_number);
}
```

Example 2: `[[uninitialized]]` on constructor

```
struct S
{
    std::string str;
    int a, b, c;

    // problematic: was the programmer aware the ints will not be initialized?
    S() = default;
};
```

With `[[uninitialized]]`:

```
struct S
{
    std::string str;
    int a, b, c;

    // OK: not initialized on purpose
    [[uninitialized(a, b, c)]] S() = default;
};
```

Example 3: forgetting member initialization

Example adapted from [Core Guideline C.48](#):

```
class X
{
    int i;
    string s;
    int j;

public:
    X() : i{666}, s{"qqq"} {}    // j is uninitialized
    X(int ii) : i{ii} {}        // s is "" and j is uninitialized
};
```

This is an example of problematic code, because in both constructors, `j` is not initialized. Marking them with `[[uninitialized(j)]]` makes it clear that it is not an oversight.

Note that this proposal does not solve the second problem in this example: that `s` has a different initialization in the two constructors. This is still an issue and should be resolved by following the guideline, `[[uninitialized]]` only helps when an initialization is deliberately left out.

Example 4: ill-formed use of `[[uninitialized]]`

```
struct S
{
    int a, b, c;

    // ill-formed, a value-initialized
    S() [[uninitialized(a, b, c)]]
    : a() {}
};

void func()
{
    // ill-formed, copy initialization
    [[uninitialized]] int i = 0;

    // ill-formed, non trivial default initialization
    [[uninitialized]] std::string str;

    // ill-formed, s is partially initialized
    [[uninitialized]] S s;
}
```

Wording

(WIP)

Edit section 8.5 [dcl.init] paragraph 12 as follows:

If no initializer is specified for an object, the object is default-initialized. When storage for an object with automatic or dynamic storage duration is obtained, the object has an *indeterminate value*, and if no initialization is performed for the object, that object retains an indeterminate value until that value is replaced (5.18): such an object is then considered uninitialized, else initialized.

Add a new section to 7.6 [dcl.attr]:

7.6.X Uninitialized attribute [dcl.attr.uninitialized]

1. The *attribute-token* **uninitialized** may only be applied to a non-member variable or a constructor. It shall appear at most once in each *attribute-list*. The uninitialized attribute specifies that an object is uninitialized (8.5) on purpose.
2. When applied to a non-member variable no *attribute-argument-clause* shall be present. If the variable is initialized but marked with the attribute, the program is ill-formed, no diagnostic required. [Note: Implementations are encouraged to provide an optional warning if a variable is uninitialized and not marked with the uninitialized attribute. —end note]
3. When applied to a constructor the *attribute-argument-clause* must have the form:

(uninitialized-id-list)

uninitialized-id-list:
mem-initializer-id...opt
uninitialized-id-list, mem-initializer-id...opt

Each *mem-initializer-id* of the *uninitialized-id-list* shall also be a valid *mem-initializer-id* of that constructors member initializer list and it shall refer to the same object. All members referred to by an *mem-initializer-id* of the *attribute-argument-clause* must be uninitialized after that constructor's member initializer list. Otherwise, the program is ill-formed, no diagnostic required. [Note: Implementations are encouraged to provide an optional warning if members are left uninitialized by a constructor and those members do not appear in the *attribute-argument-clause* of the constructor's unused attribute or the constructor is not marked with the attribute. —end note]

4. [Example:

```
struct base
{
```

```

    int a, b, c;

    // constructor does not initialize any member
    [[uninitialized(a,b,c)]] base() = default;

    // ill-formed: a is value initialized
    base(int i, int j) [[uninitialized(a)]]
    : a(), b(i), j(i) {}
};

struct derived
: base
{
    std::string str;
    int x, y;

    // constructor only initializes str and y
    derived(int i) [[uninitialized(base, x)]]
    : str("foo"), y(i) {}

    // ill-formed: str has non trivial default constructor
    derived(int i, int j) [[uninitialized(str)]]
    : base(i, j), x(i), y(j) {}
};

struct uninitialized t {};

struct s
{
    float a, b;

    // constructor does not initialize anything
    s(uninitialized t) [[uninitialized(a,b)]]
    {}
};

void func()
{
    // not initialized
    [[uninitialized]] int i;
    // ill-formed: copy initialized
    [[uninitialized]] int j = 0;

    // not initialized, default constructor is trivial
    [[uninitialized]] base b;
    // ill-formed: default initialized with non trivial constructor
    [[uninitialized]] std::string str;

    // ill-formed: direct initialized (even though members only partially)
    [[uninitialized]] derived d(42);
    // ill-formed: direct initialized (even though members only partially)
    [[uninitialized]] s obj(uninitialized{});
}

```

—end example]